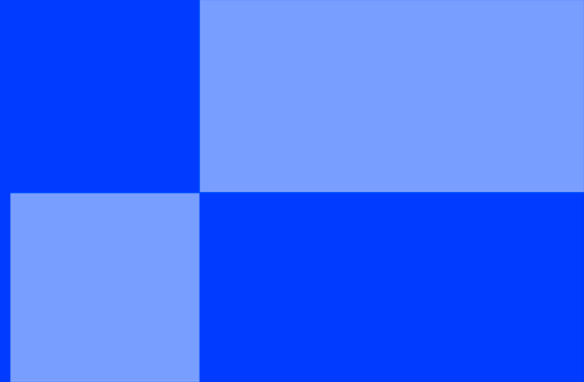


{EPITECH}



VIGIL



< RATRAPAGE />

VIGIL

Context

Teams that deploy to production face two types of events:

- ✓ **Planned events** — a release is prepared, its steps are validated one by one before reaching production.
- ✓ **Unplanned events** — something goes wrong, and it must be triaged, escalated, assigned and resolved quickly.

VIGIL is a collaborative operational control room that handles both realities in real time. Teams coordinate their **Releases** (planned deployments, validated step by step) and their **Incidents** (detected problems, triaged and resolved). The two are connected: a Release can automatically trigger an Incident, and an active Incident can block an ongoing Release.

This project simulates real startup conditions: tight time constraints, technical decisions to make and own, delivery of a functional product, and presentation to a jury.

T-DEV-600 exemption clause

If you have **already validated** the CI/CD or Internationalization achievements during your T-DEV-600 attempt, you are not required to re-implement them in VIGIL. The achievements concerned are:

- ✓ `repo_cicd` - if validated, no GitHub Actions pipeline is required from you in Phase 2
- ✓ `web_multilingual` and `desktop_multilingual` - if validated, the FR/EN internationalization is not required in Phase 2 or Phase 3

To benefit from the exemption, you must **declare it at the project kickoff** and provide evidence of the prior validation (link to the T-DEV-600 repository, screenshot of the achievement grid, or any artifact accepted by the pedagogical team).

The exemption only covers the specific achievements listed above. All other achievements of T-DEV-600 must be covered by VIGIL deliverables regardless of your prior T-DEV-600 attempt.

Required Stack

Component	Technology
Application server	Rust (Axum) ou NodeJS
Web client	Next.js
Desktop client	Tauri or Electron
Persistence	PostgreSQL or SQLite
Real-time	WebSockets
Containerization	Docker Compose
CI/CD	GitHub Actions



The stack listed above is **imposed**. Your README must document the stack you use. For the three components where a choice is left to you — **Application server** (Rust or NodeJS), **desktop client** (Tauri or Electron) and **persistence** (PostgreSQL or SQLite) — your README must additionally **justify your choice** with a short paragraph explaining why you retained one option over the other in the context of VIGIL. Once decided, your stack does not change for the duration of the project.

The software suite is composed of three parts:

- ✓ An **application server** — all business logic
- ✓ A **web client** (Next.js) — main interface for configuration and supervision
- ✓ A **desktop client** (Tauri or Electron) — standalone native application



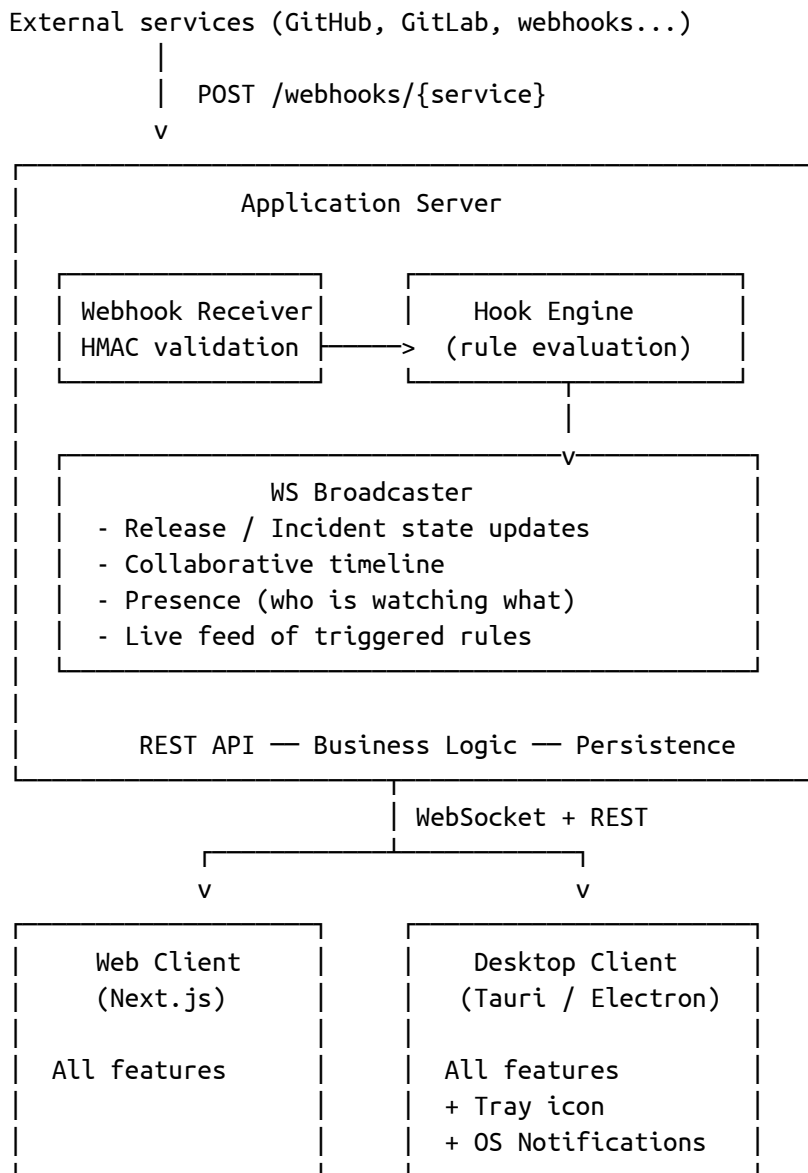
No business logic should live in the clients. They display and forward requests to the application server.

Requirements

Before starting take care of this section:

- ✓ Your application accepts **multiple simultaneous connections**
- ✓ Each user must register before they can use the application
- ✓ The client and the server must use socket
- ✓ You need to produce a document specification of your socket usage
- ✓ The server must enforce permissions and return appropriate error responses
- ✓ Your application respects the **UX & Interface** requirements detailed in the dedicated section below
- ✓ **All** of your data should be persisted

Architecture



Client Parity

The web client and the desktop client expose **exactly the same features**. The only difference comes from the native nature of the desktop client: it remains active in the background via a tray icon when the window is closed, and it triggers native OS notifications on assignment or critical state changes even when the main window is closed.

UX & Interface

VIGIL is presented as an operational control room. Beyond raw functionality, both clients (web and desktop) must demonstrate a **professional, consistent, and accessible user experience**.

Visual consistency

The interface obeys uniform rules across all screens:

- ✓ A documented color palette (3 to 5 primary colors) with explicit usage rules (primary action, secondary action, success, warning, danger)
- ✓ A consistent typographic hierarchy (at least 3 distinct text levels: title, subtitle, body)
- ✓ A consistent spacing grid

Two different screens performing similar actions must look alike.

Accessibility

The interface follows accessibility minimums documented by the student in `UI_GUIDELINES.md`. The minimums applied must include at least:

- ✓ **Keyboard navigation** on all primary actions (create Incident, acknowledge, escalate, validate Release step)
- ✓ **Explicit labels** on all form fields (no placeholder-only inputs)
- ✓ **Color is never the only signal** — every state (Incident state, severity level) is conveyed by color **and** icon **and** text, to remain readable for color-blind users

The student documents in `UI_GUIDELINES.md` the level of accessibility they target and how they verified compliance. A public reference standard may be cited if the student wishes. A higher level of accessibility contributes to a stronger evaluation on the `uiux_quality` achievement.

Information hierarchy

Critical actions are visually distinct from secondary actions. Each lifecycle state and severity level has a recognizable visual representation that an operator can identify in under one second:

- ✓ Incident states: `open`, `acknowledged`, `escalated`, `resolved`
- ✓ Severity levels: `low`, `medium`, `high`, `critical`
- ✓ Release states: `created`, `in_progress`, `completed`, `cancelled`, `blocked`

Dark patterns

The interface must not use [dark patterns](#). Specifically:

- ✓ **All destructive actions** (delete, kick, ban, cancel Release, transfer Manager role) must require **explicit confirmation** via a dedicated dialog naming the affected resource
- ✓ No confirmation inversion (e.g. "Click here to not unsubscribe")
- ✓ No critical options hidden behind non-obvious UI affordances

Deliverable

A `UI_GUIDELINES.md` file at the project root, **1 to 2 pages**, documenting:

- ✓ The color palette and usage rules
- ✓ The mapping of each lifecycle state and severity to its visual representation
- ✓ The list of reusable UI components and their variants
- ✓ The dark patterns identified during design and how they are avoided
- ✓ **At least 2 annotated screenshots** illustrating the rules in practice

This file is the contract used by the jury to evaluate the UX requirements during the demo.

Objectives

You must build VIGIL in **3 successive phases**.



Each phase must be completed before moving on to the next one.

Priorities & Phase Validation

Each phase contains a **core** scope and an **extended** scope.

- ✓ **Core** items are mandatory to validate the phase and unlock the next one. A phase is not considered complete if any core item is missing.
- ✓ **Extended** items are evaluated at the final jury. They contribute to the overall grade but do not block phase progression.

The student is expected to deliver the core scope of each phase before working on extended items. Working on Phase 2 extended items while Phase 1 core is incomplete is a methodological mistake.

The detailed core / extended split for each phase is given in the phase's introduction below.

Phase 1: Core Feature

Core scope (mandatory to validate Phase 1):

- ✓ Email/password authentication, `GET /me`, sign out with token invalidation
- ✓ Teams with 3-role system (Observer / Responder / Manager), invitation code, Manager role transfer
- ✓ Incidents with full lifecycle (open → acknowledged → escalated → resolved), severity levels, real-time collaborative timeline
- ✓ Persistence of all data
- ✓ WebSockets: `incident_state_changed`, `incident_escalated`, `incident_assigned`, `timeline_entry_added`, `presence_update`
- ✓ Automatic client-side reconnection

Extended scope (evaluated at final jury):

- ✓ OAuth2 sign-in (GitHub recommended)
- ✓ Releases with full lifecycle and automatic blocking by linked Incidents
- ✓ Member Moderation (kick, temporary ban, permanent ban)
- ✓ Timeline Entry Editing
- ✓ Private Messages
- ✓ Timeline Reactions
- ✓ WebSockets: `member_kicked`, `member_banned`, `timeline_entry_edited`, `private_message_received`, `reaction_added`, `reaction_removed`, `release_step_validated`, `release_state_changed`

Your application must allow complete management of Teams, Releases and Incidents in real time.

Authentication & User Management

- ✓ Sign up via **email / password**
- ✓ Sign in via **OAuth2** (GitHub recommended) — extended scope
- ✓ `GET /me` endpoint returning current user information
- ✓ Sign out with token invalidation
- ✓ The server returns appropriate error codes on all protected routes (401, 403, 404...)

Teams

A user can belong to multiple **Teams**. A Team is the shared workspace in which Releases, Incidents and rules are managed.

Each Team has a **3-role system**:

Role	Permissions
Observer	View Releases and Incidents, read the timeline, see member presence
Responder	Everything an Observer can do + acknowledge an Incident, validate a Release step, add a timeline entry
Manager	Everything a Responder can do + create / close / cancel Releases and Incidents, assign Responders, configure rules, manage members and their roles



A Team has only one Manager. They can transfer this role to another member. A Manager cannot leave a Team without transferring their role first.

Access to a Team is granted via an **invitation code** generated by the Manager.

Member Moderation

A **Manager** can moderate the Team membership:

- ✓ **Kick** — Remove a member from the Team. The user no longer appears in the member list and loses access to the Team's resources. The user can rejoin later via a new invitation code.
- ✓ **Temporary ban** — Forbid a user from joining the Team until a given expiration date. While banned, the user cannot accept an invitation code, even a valid one. The ban automatically lifts at expiration.
- ✓ **Permanent ban** — Same as temporary ban, with no expiration. A permanent ban can only be lifted by an explicit Manager action.

When a member is kicked or banned, their existing timeline entries, acknowledgments, and Release step validations **remain attributed to them** in the Team's history. Moderation actions do not rewrite the past.

The Manager cannot kick, ban, or transfer the Manager role to themselves.

Private Messages

Any member of a Team can send a **direct message** to any other member of the same Team, regardless of their roles. Private messages are strictly bilateral (1-to-1), never grouped, and never tied to an Incident or Release.

- ✓ A user can open a private conversation with any member of any Team they belong to
- ✓ Conversation history is persisted and accessible to both participants
- ✓ A reasonable message length limit must be enforced server-side (around 2000 characters is recommended); the chosen limit is documented in the README
- ✓ New messages are delivered in real time to the recipient via WebSocket

Private messages cannot be sent between users who do not share at least one Team.

Releases

A **Release** represents a planned deployment, coordinated by the team in real time.

Lifecycle:

```

created > in_progress > completed
                        |—————> cancelled
                        |—————> blocked (active linked Incident)

```

Each Release is composed of configurable **sequential steps**. A step must be validated before the next one becomes available. Standard step examples: `build`, `staging`, `go_no_go`, `production`.

If an Incident is created and linked to an `in_progress` Release, that Release automatically moves to `blocked` until the Incident is resolved.

Incidents

An **Incident** represents an unplanned problem that requires a coordinated response.

Lifecycle:

```

open > acknowledged > escalated > resolved

```

Severity levels: `low` / `medium` / `high` / `critical`

Each Incident has a **timeline** — a timestamped log of entries visible to all connected members, updated in real time.

A Manager can **assign a Responder** to an Incident. The assigned Responder receives an immediate notification via WebSocket.

Timeline Entry Editing

The author of a timeline entry can edit its content after publication. The edited entry retains its original `at` timestamp and gains an `edited_at` timestamp, both visible to other members.

Only the original author can edit their own entries. Managers cannot edit entries authored by other members.

Timeline Reactions

Members can react to timeline entries with a fixed set of emojis exposed by the server. Reactions are aggregated and visible to all Team members in real time.

- ✓ The list of available emojis is exposed via `GET /reactions/available` and is identical for all clients
- ✓ A fixed set of emojis is defined by the server. A set of 5 to 8 emojis is recommended (for example: `+1`, `-1`, `eyes`, `warning`, `check`, `fire`). The chosen set is documented in the README
- ✓ Each user can add or remove their own reaction on any timeline entry
- ✓ A user cannot add the same emoji twice on the same entry
- ✓ Reactions are visible only on timeline entries of Incidents (not on Release step validations, not on private messages)

WebSockets – Phase 1

The WebSocket connection is **one per client**. It must carry at minimum the following events by the end of this phase:

```
{ "type": "incident_state_changed", "incident_id": "uuid", "new_state": "acknowledged", "by": "alice" }
{ "type": "incident_escalated", "incident_id": "uuid", "new_severity": "critical", "by": "bob" }
{ "type": "incident_assigned", "incident_id": "uuid", "assigned_to": "alice" }
{ "type": "timeline_entry_added", "incident_id": "uuid", "entry": { "content": "...", "author": "alice", "at": 1718000000 } }
{ "type": "release_step_validated", "release_id": "uuid", "step": "staging", "by": "alice" }
{ "type": "release_state_changed", "release_id": "uuid", "new_state": "blocked" }
{ "type": "presence_update", "resource_id": "uuid", "resource_type": "incident", "watchers": ["alice", "bob"] }
{ "type": "member_kicked", "team_id": "uuid", "member": "alice", "by": "bob" }
{ "type": "member_banned", "team_id": "uuid", "member": "alice", "until": 1718000000, "by": "bob" }
{ "type": "timeline_entry_edited", "incident_id": "uuid", "entry_id": "uuid", "new_content": "...", "edited_at": 1718000000 }
{ "type": "private_message_received", "from": "alice", "to": "bob", "content": "...", "at": 1718000000 }
{ "type": "reaction_added", "incident_id": "uuid", "entry_id": "uuid", "emoji": "+1", "by": "alice" }
{ "type": "reaction_removed", "incident_id": "uuid", "entry_id": "uuid", "emoji": "+1", "by": "alice" }
```

Notes:

- ✓ For `member_banned`, `until` is `null` for permanent bans.
- ✓ Private message events (`private_message_received`) are delivered only to the sender and the recipient, not broadcast to the Team.



Automatic client-side reconnection is mandatory from this phase onwards.

Phase 2: Automation & Professionalization

Core scope (mandatory to validate Phase 2):

- ✓ Rule engine with at least 1 external service in Action, VIGIL plus 1 additional service in REAction, and 1 complete working rule
- ✓ `/about.json` exposing the service catalog dynamically, including the SHA-256 kickoff token
- ✓ WebSockets: `rule_triggered`, `rule_failed`
- ✓ Server-side encrypted storage of OAuth2 tokens and personal tokens
- ✓ CI/CD pipeline functional (waived if validated in T-DEV-600)

Extended scope (evaluated at final jury):

- ✓ Internationalization FR/EN on the web client (waived if validated in T-DEV-600)
- ✓ Linter, formatter, coverage report integrated in CI (see Quality Thresholds)
- ✓ Additional services beyond the minimum (GitLab, Discord, HTTP, Email, Timer, Generic Webhook)
- ✓ HOWTOCONTRIBUTE.md detailed and up to date

Your application must integrate the automation engine and demonstrate professional practices.

Action → REAction Rule Engine

External events automatically trigger actions inside VIGIL.

Services & Connection

Users connect third-party services from their profile. The list of available services is **exposed dynamically** by the server via `/about.json` — clients never hard-code any service.

Each service is authenticated via **OAuth2** or a **personal token**. Tokens are stored server-side, never in plain text.

Action Components

Service	Available events
GitHub	CI workflow failed, CI workflow succeeded, new tag pushed, PR merged
GitLab	Pipeline failed, pipeline succeeded, tag created
Generic Webhook	Arbitrary JSON payload on <code>/webhooks/generic/:id</code>
Timer	At time <code>HH:MM</code> every day, every <code>X</code> minutes

REAction Components

Service	Available actions
VIGIL	Create an Incident, validate a Release step, block a Release, escalate an Incident
Discord	Send a message on a channel
HTTP	Send a POST request to an external URL
Email	Send an email to a configured address



The tables above list examples. The minimum required scope for the rule engine is:

- ✓ **1 external service** in Action (your choice among GitHub, GitLab)
- ✓ **VIGIL plus 1 additional service** in REAction (your choice among Discord, HTTP, or Email)
- ✓ **At least 1 complete and working rule** demonstrable end-to-end during the jury

You are free to implement more services. The evaluation focuses on the quality and robustness of one complete chain, not on the breadth of the catalog. The `HOWTOCONTRIBUTE.md` file must describe how to add new services regardless of how many you have implemented yourself.

Rule Composition

```
{
  "name": "CI failure > Critical Incident",
  "enabled": true,
  "trigger": {
    "service": "github",
    "event": "workflow_run",
    "filters": {
      "conclusion": "failure",
      "repository": "my-org/my-repo"
    }
  },
  "reaction": {
    "type": "vigil_create_incident",
    "payload": {
      "title": "CI broken on {{repository.name}}",
      "severity": "high",
      "body": "Workflow {{workflow.name}} failed - [View run]({{run.url}})"
    }
  }
}
```

WebSockets — Phase 2 additions

```
{ "type": "rule_triggered", "rule_name": "CI failure > Incident", "result": "incident_created",
  "incident_id": "uuid" }
{ "type": "rule_failed", "rule_name": "CI failure > Incident", "error": "service_unavailable" }
```

/about.json file

```
{
  "client": { "host": "10.0.0.1" },
  "server": {
    "current_time": 1718000000,
    "services": [
      {
        "name": "github",
        "actions": [
          { "name": "workflow_run_failed", "description": "A CI workflow run completes with a failure conclusion" }
        ],
        "reactions": [
          { "name": "create_incident", "description": "Create a VIGIL incident with configurable severity and title" }
        ]
      }
    ]
  }
}
```



You must add a token field in your **about.json file** which is a SHA-256 hash. The content of the token will be displayed during the kick-off

Quality Thresholds

The following thresholds are inherited from the original T-JSF-600 and T-DEV-600 modules. They are evaluated by the jury and contribute to the `tests_unit`, `tests_coverage`, `code_style`, and `code_maintainability` achievements.

Test coverage

- ✓ The source code reaches a minimum of **70% line coverage** on automated tests (inherited from `tests_unit`, both modules)
- ✓ A coverage report is **published as a CI artifact** on every merge to `main` (inherited from `tests_coverage`, T-DEV-600)
- ✓ The report demonstrates that **branch coverage extends beyond the main flow** — error paths and conditional branches are also tested (inherited from `tests_coverage`, T-JSF-600)
- ✓ Coverage is measured by a tool of the student's choice (e.g. `cargo tarpaulin` for the server, the test runner's built-in coverage for the clients)

Linting and formatting

The following commands must pass in CI on every branch push:

- ✓ **Rust:** `cargo clippy` passes without errors, `cargo fmt --check`
- ✓ **TypeScript:** ESLint with a configuration documented in the README passes without errors, `prettier --check`

A linter passing in CI is how the `code_style` achievement (consistent coding standards) is operationally verified.

Code maintainability

The `code_maintainability` achievement is evaluated by the jury against the criteria of the original brief: human-readable names, atomicity of functions, clear code structure, clean syntax.

To support this evaluation, the **README must document the architectural decisions**:

- ✓ Where business logic lives (handlers, services, domain models)
- ✓ Where HTTP routes are wired
- ✓ Where persistence is accessed
- ✓ Where the WebSocket broadcaster lives

This section is the entry point the jury uses to navigate the codebase.

Internationalization (i18n)

Both clients must provide an interface available in **French and English**. The language is configured in the user profile and persisted server-side.

The following must be translated:

- ✓ All interface labels (buttons, labels, titles)
- ✓ Release and Incident states (open, acknowledged, resolved...)
- ✓ Severity levels (low, medium, high, critical)

CI/CD

A **GitHub Actions** pipeline must be set up with at minimum:

- ✓ **On every branch push**: linting (clippy, ESLint, fmt, prettier passing without errors) + unit tests
- ✓ **On merge to main**: full build + integration tests + coverage report published as artifact
- ✓ **On tag creation** (v*.*.*) : build release artifacts (desktop binary + Docker image)



Set up your pipeline early. A broken CI in week 10 is a planning problem, not an accident.

Phase 3: Desktop Application

Core scope (mandatory to validate Phase 3):

- ✓ Installable desktop application on the chosen target OS, with all VIGIL features functional
- ✓ Native OS notifications on the three required triggers (assignment, critical severity, blocked Release)
- ✓ Docker Compose configuration exposing the desktop binary via the web client
- ✓ README.md complete

Extended scope (evaluated at final jury):

- ✓ WEBSOCKET_SPEC.md complete and up to date
- ✓ HOWTOCONTRIBUTE.md complete and up to date
- ✓ UI_GUIDELINES.md with at least 2 annotated screenshots (see UX & Interface)
- ✓ Desktop binary build verified in CI

Your server must be accessible through a **native desktop application** built with Tauri or Electron.

Expected Features

- ✓ Installable application on **one OS of your choice** (Windows, macOS or Linux), documented in the README
- ✓ Complete graphical interface allowing use of **all VIGIL features**
- ✓ Multilingual FR/EN (i18n shared with the web client)
- ✓ **Native OS notifications** triggered on:
 - The user being assigned to an Incident
 - An Incident reaching **critical** severity
 - A Release being blocked by an Incident

Technology Requirement

The desktop application must be built with either:

- ✓ **Tauri** (recommended if the backend is in Rust)
- ✓ **Electron**

Build & Delivery (Docker)

The `docker-compose.yml` file at the root of the project must describe at minimum:

- ✓ `server` — application server on port **8080**
- ✓ `client_web` — Next.js client on port **8081**
- ✓ `client_desktop` — binary build service, shares a volume with `client_web`
- ✓ `db` — database

`client_web` depends on both `server` AND `client_desktop`.

The `client_web` exposes the desktop binary at one of:

- ✓ GET `http://localhost:8081/client.exe` (if Windows is your target OS)
- ✓ GET `http://localhost:8081/client.AppImage` (if Linux is your target OS)
- ✓ GET `http://localhost:8081/client.dmg` (if macOS is your target OS)

The choice of target OS must be documented in the README.

Documentation

README.md

- ✓ Global architecture with diagram
- ✓ Stack documentation, including justification of the two choices left to the student (desktop client, persistence)
- ✓ Installation and local setup instructions
- ✓ Full REST API documentation
- ✓ Commented database schema
- ✓ Architecture section navigating the codebase (where business logic, handlers, persistence, WS broadcaster live)

WEBSOCKET_SPEC.md

- ✓ Each WebSocket event type
- ✓ Full payload structure
- ✓ Trigger conditions
- ✓ Target clients (who receives what)

HOWTOCONTRIBUTE.md

- ✓ How to add a new service
- ✓ How to add a new Action
- ✓ How to add a new REAction
- ✓ How to add a new WebSocket event

UI_GUIDELINES.md

- ✓ Color palette with usage rules
- ✓ State-to-visual mapping (Incident state, severity, Release state)
- ✓ Reusable UI components and variants
- ✓ Dark patterns identified and mitigation
- ✓ At least 2 annotated screenshots



All documentation must be written in **Markdown**.

Keynote

You will present VIGIL to a jury.

Expected structure:

1. **Introduction** — the problem VIGIL solves, why this product exists
2. **Technical architecture** — your stack choices and why you made them
3. **Methodology** — how you organized 12 weeks of solo development
4. **Live demo** — no safety net, on a stable environment prepared in advance
5. **Code walkthrough** — one feature of your choice explained in depth
6. **Q&A** — The jury will ask technical and organizational questions on your work



Prepare your demo on a **stable, isolated environment**. A demo that crashes penalizes more than a missing feature.

Specific Expectations

What We Want to See

A **product that works** — fewer features that work are always better than many that crash. **Professionalism** in the code, tests on critical paths, documentation, and CI. **Autonomy** in your organization as a solo professional. **Thoughtfulness** in your technical decisions and the ability to defend them. A **flawless demo** prepared in advance.

What We Don't Want to See

Untested code or code that doesn't compile. Applications that crash during the demo. Monolithic architecture with no separation of concerns. Documentation that is absent or AI-generated without review. Improvised presentations.

v 1.3.0

{EPITECH}